

[Dashboard](#) / [My courses](#) / [CS F301](#) / [Object Orientation: Abstract Data Types](#) / [OOP Quiz](#)

Status	Finished
Started	Monday, 28 September 2026, 10:02 PM
Completed	Monday, 28 September 2026, 10:42 PM
Duration	40 mins
Grade	Not yet graded

Question **1**

Complete

Marked out of 8.00

For each stack, start at its top, form a string using successive top-down elements, and stop at the first null byte '\0'. What comparison keys are produced and what is the ascending order?

Input / scenario data

S1 top -> 'c', 'a', 't', '\0', 'x'

S2 top -> 'c', 'a', 'r', '\0'

S3 top -> 'd', 'o', 'g', '\0'

Estimated time: 2-3 minutes

Each stack is read from the top, and characters are appended until the first null byte '\0', which acts as the string terminator.

- For Stack S1: string is formed as [c,a,t]. The element 'x' is not a part of the key and is never read because it appears after/below the string terminator '\0'.
- For Stack S2: string is formed as [c,a,r].
- For Stack S3: string is formed as [d,o,g].

Thus, keys produced:

- S1: "cat"
- S2: "car"
- S3: "dog"

Comparing the keys produced lexicographically, ascending order: S2 ("car") < S1 ("cat") < S3 ("dog").

Question **2**

Complete

Marked out of 4.00

In the context of the professor's module-independence exercise, give one potential modularity advantage of Design B over Design A.

Code / relevant fragment

```
// Design A
class MyList extends Storage {
    ...
}
```

```
// Design B
class MyList {
    private Storage storage;
    ...
}
```

Estimated time: 2-3 minutes

- The Design A suffers from the Fragile Base Class problem as mentioned in the "Goodbye, OOP" article. MyList is dependent on Storage's implementation. All public methods of Storage also become available in MyList, so users can access things that may not be intended for use. An internal change in Storage can break MyList without MyList's code changing.
- In Design B (private Storage storage), MyList uses composition. MyList contains a Storage object and uses its methods when needed. Users only see the methods provided by MyList. This means Storage can be changed or replaced more easily without affecting the users of MyList.

Thus, the advantage is that Design B keeps the two modules separate by hiding Storage behind MyList's own interface.

Question **3**

Complete

Mark 2.00 out of 2.00

What do size and count represent in the following MyDictionary state?

Code / relevant fragment

```
private Record[] records;  
private int size = 0;  
private int count = 0;
```

```
MyDictionary(int initialSize) {  
    records = new Record[initialSize];  
    size = initialSize;  
    count = 0;  
}
```

Estimated time: 30 seconds

Select one:

- ☐ A. size is occupied records; count is array capacity.
- ☐ B. Both always have the same value.
- ☒ C. size is maximum capacity; count is the current number of stored records.
- ☐ D. size is the key range; count is the number of comparisons.

Question **4**

Complete

Marked out of 8.00

Replace the switch logic below with code that obeys the actual Comparable.compareTo() contract.

Code / relevant fragment

```
switch(comparisonResult) {  
    case 0: return Comparison.MATCHING;  
    case -1: return Comparison.PREDECESSOR;  
    case 1: return Comparison.SUCCESSOR;  
    default: return Comparison.INCOMPARABLE;  
}
```

Estimated time: 2 minutes

```
if (comparisonResult == 0)  
{  
    return Comparison.MATCHING;  
}  
else if (comparisonResult < 0)  
{  
    return Comparison.PREDECESSOR;  
}  
else  
{  
    return Comparison.SUCCESSOR;  
}
```

Question **5**

Complete

Marked out of 8.00

The class exercise asks students to make a single instance of a class deviate in a call to a method from all other instances of the class. Without modifying every Stack, give one Java-oriented way of making only one created object use different remove() behavior. What OO mechanism is actually being used?

Code / relevant fragment

```
class Stack {  
    public Object remove() { ... }  
}
```

Estimated time: 3 minutes

Create that one object as an **anonymous inner class** that **overrides** remove():

```
Stack s = new Stack() { @Override public Object remove() { /* different behavior */ } };
```

This changes the behavior only for this particular object. The Stack class and other Stack objects remain unchanged.

OO Mechanism: This uses **inheritance** and **method overriding**. Java creates an anonymous subclass of Stack, and dynamic dispatch makes the overridden remove() run for that object.

Question **6**

Complete

Mark 2.00 out of 2.00

Which statements correctly characterize the fragile-base-class example in the article?

Estimated time: 1 minute

Select all correct answers. Each correct selection earns one-third of the marks; selecting the incorrect option deducts all available marks. The minimum score is zero.

Select one or more:

- ☒ A. The subclass may break even though its own source code was not modified.
- ☒ B. A base-class internal implementation change can alter which overridden method gets called.
- ☐ C. The public behavior of the base class itself must necessarily become incorrect.
- ☒ D. The subclass may depend on implementation details of the superclass.

Question **7**

Complete

Mark 2.00 out of 2.00

What does the current implementation actually do with ordering?

Code / relevant fragment

```
private Ordering ordering = Ordering.NONE;

enum Ordering {
    NONE, ASC, DESC, PRIQASC, PRIQDESC, BST
}
```

Estimated time: 30 seconds

Select one:

- ☐ A. It sorts all inserts automatically.
- ☐ B. It selects a binary search tree.
- ☐ C. It changes the implementation of get().
- ☒ D. Nothing beyond initializing the field.

Question **8**

Complete

Marked out of 2.00

Explain "Contain-and-Delegate" in one or two sentences using the article's Copier, Scanner, and Printer example.

Estimated time: 1 minute

Contain-and-Delegate means reusing behavior by containing other objects instead of inheriting from them. From the Goodbye, OOP article, Copier contains a Scanner and a Printer, and its start() method delegates the work to printer.start(), avoiding the problem of choosing between the two inherited start() methods. This makes the code more flexible because Copier can choose which object's behavior to use.

Question **9**

Complete

Marked out of 4.00

Assume the missing brace has been repaired but the method remains as shown. The array begins with keys [5, 9, 12]. What does get(9) return?

Code / relevant fragment

```
public Record get(int key) {  
    for (Record record : records) {  
        if(record.key == key)  
            return record;  
        return null;  
    }  
    return null;  
}
```

Estimated time: 1 minute

get(9) returns **null**.

The return null is inside the for loop. The method exits after checking only the first record (key 5 not equal to 9), and returns null. The record with key 9 is never checked.

Question **10**

Complete

Mark 2.00 out of 2.00

Which statement most closely describes the Banana-Monkey-Jungle problem as discussed in the supplied article?

Estimated time: 45 seconds

Select one:

- ☐ A. Every subclass must contain a monkey object.
- ☒ B. Reusing one desired class may force us to bring along its parent hierarchy and other dependencies.
- ☐ C. Composition always creates cyclic dependencies.
- ☐ D. Interfaces require every implementation class to be copied.

Question **11**

Complete

Marked out of 4.00

True or False: Because AbstractRecord exists, this Dictionary can already store arbitrary subclasses of AbstractRecord. Justify your answer.

Code / relevant fragment

```
private Record[] records;
```

```
public void put(Record record) {  
    ...  
}
```

Estimated time: 1 minute

FALSE.

The dictionary uses Record[] and put(Record record), so it only accepts objects that are Record (or subclasses of Record). A different subclass of AbstractRecord cannot be passed to put() unless the dictionary is changed to use AbstractRecord instead.

Question **12**

Complete

Marked out of 4.00

The original statement is changed from `records[count++] = record;` to `records[++count] = record;`. Assume `count` initially equals 0 and the array has length 3. What immediately changes if nothing else is modified?

Code / relevant fragment

```
// Original
records[count++] = record;
```

```
// Changed
records[++count] = record;
```

Estimated time: 2 minutes

With `++count`, `count` increases before it is used as the array index. So the first record is stored at `records[1]` instead of `records[0]`. This means the third insert tries to use `records[3]`, which is outside the array and causes an `ArrayIndexOutOfBoundsException`. Therefore, only **2 records can be stored** in the array of size 3.

◀ [Tasks for the End of the OO Module](#)

[Jump to activity](#)

[The Dictionary bundle with some modifications ...](#) ▶